QUANTITATIVE FINANCE STUDY GUIDE

State Space Models in Quantitative Finance

Latent states, the Kalman filter, dynamic hedge ratios and pairs trading, with interview preparation

- Intuition → mathematics → worked example → Python → interpretation → limitations
- Kalman filtering, smoothing, maximum-likelihood estimation and diagnostics
- Latent trends, time-varying regression, pairs trading and a full out-of-sample case study with a structural break
- Ten interview questions, five numerical / coding problems, formula sheet and glossary

How the numbers in this guide were produced

Every number, table value and chart was produced by executing the Python code shown (NumPy, SciPy and Matplotlib, with fixed random seeds). The Kalman filter is implemented from scratch for transparency and was cross-checked against the statsmodels implementation. **All price data are synthetic**: they are generated from known models so that estimates can be compared with the truth, and so no result here is evidence that a strategy would make money on real markets.

This guide is educational material, not investment advice.

October 2026

Contents

1. Introduction and Intuition 3
1.1 What a state space model is 3
1.2 Analogies 3
1.3 Why finance needs latent-variable models 3
1.4 How SSMs differ from familiar tools 3

2. Mathematical Foundations 4
2.1 The linear Gaussian state space model 4
2.2 Why financial observations are noisy proxies 5
2.3 A first numerical example: a hidden state generates data 5

3. The Kalman Filter 6
3.1 Idea: predict, compare, correct 6
3.2 The equations 6
3.3 Worked example: a scalar model, step by step 7
3.4 Interpretation on the synthetic series 8
3.5 Assumptions and when to go beyond the standard filter 8

4. Smoothing, Forecasting, and Model Estimation 9
4.1 Filtering, smoothing and forecasting 9
4.2 Rauch–Tung–Striebel smoother 9
4.3 Forecasting and the two kinds of uncertainty 10
4.4 Initialisation and unknown parameters 10
4.5 Diagnostics and parameter sensitivity 11

5. Financial Applications 12
5.1 Latent trend estimation 12
5.2 Time-varying regression and dynamic hedge ratios 13
5.3 Pairs trading 13
5.4 Market regimes 13
5.5 Volatility estimation 13
5.6 Signal extraction 14

6. Important Model Variants 14

7. Practical Python Implementations 15
7.1 A scalar Kalman filter 15
7.2 Estimating a latent trend 16
7.3 A time-varying hedge ratio 16
7.4 A basic pairs-trading research example 17

8. Integrated Quantitative Trading Case Study 18
8.1 Model fit and signal 19
8.2 Evaluation: stability and economic usefulness 19
8.3 Sensitivity, costs and model monitoring 20
8.4 When this model fails 21

9. Limitations and Common Mistakes 21
9.1 Model risk versus estimation uncertainty 22

10. Interview Revision and Formula Sheet 22
10.1 Ten interview questions 22
10.2 Five numerical, mathematical and coding questions 23
10.3 Formula sheet 23
10.4 Glossary 24
10.5 Further reading and references 24

1. Introduction and Intuition

1.1 What a state space model is

Many quantities that matter in finance cannot be observed directly: the "true" value of a stock stripped of microstructure noise, the persistent trend in a price, the current hedge ratio between two assets, the volatility regime. What we see are **noisy measurements** of them. A **state space model (SSM)** separates the problem into two layers: a **hidden (latent) state** x_t that evolves over time according to some dynamics, and an **observation** y_t that is a noisy function of that state. The goal is to infer the hidden state from the observations as they arrive.

The distinction between *observable data* (prices, returns, spreads) and *latent states* (trend, hedge ratio, regime) is the heart of the subject. It is also why an SSM is a **model**, whereas the **Kalman filter** (Chapter 3) is an **algorithm** that computes the best estimate of the state under the model's linear-Gaussian assumptions. Confusing the two is a classic interview error.

1.2 Analogies

Tracking a ship. A radar returns the ship's position with random error each sweep. The ship's true position and velocity are hidden, but physics says it moves smoothly. Combining the physics (a prediction) with each noisy radar blip (an observation) gives a better position estimate than either alone. The Kalman filter was developed for exactly this kind of problem (Kalman, 1960). **A thermostat** in a noisy room works the same way: the true temperature drifts slowly, the sensor jitters, and a sensible controller averages them with weights that reflect how much it trusts each.

In finance the "ship" is the persistent component of a price or a relationship, the "radar" is the noisy market quote, and the "physics" is an assumption about how the hidden component evolves (for example, that it follows a random walk).

1.3 Why finance needs latent-variable models

- **Prices** are efficient values plus bid–ask bounce, discreteness and temporary impact: the observed price is a noisy proxy for a smoother underlying value.
- **Trends and signals** are hidden: a moving average gives one crude estimate, but it cannot say how uncertain it is or adapt its speed to the noise level.
- **Relationships drift.** A hedge ratio or factor exposure estimated over a fixed window is stale the moment it is computed; an SSM lets it evolve with quantified uncertainty.
- **Volatility and regimes** are never directly observed; they are inferred from returns.

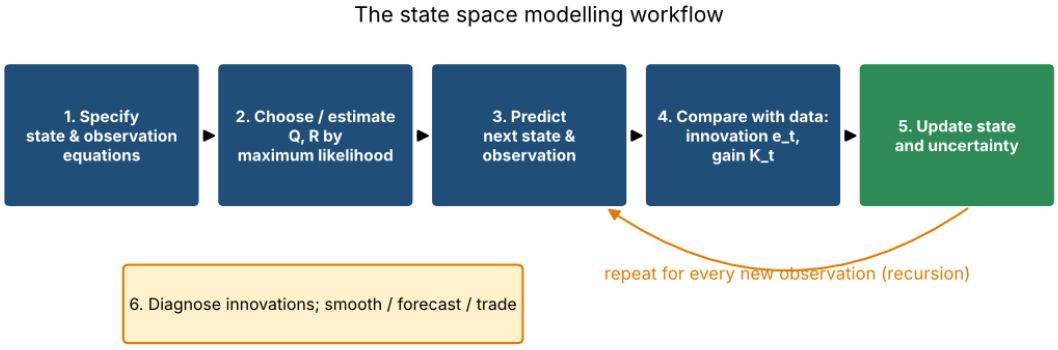


Figure 1. The state space modelling workflow: specify the model, estimate its noise parameters, then recursively predict, compare with the data and update. Diagnostics of the prediction errors close the loop.

1.4 How SSMs differ from familiar tools

Table 1. State space models compared with regression, ARIMA and moving averages

Tool	What it assumes	Strength	Limitation relative to an SSM
Ordinary regression	Fixed coefficients over the whole sample	Simple, interpretable	Cannot represent drifting relationships; no recursive updating

Tool	What it assumes	Strength	Limitation relative to an SSM
Rolling regression	Constant coefficients inside a window	Adapts crudely	Window length arbitrary; noisy and abrupt; discards old data entirely
Moving average	No model; equal (or chosen) weights	Transparent	Lags trends; weights not tied to noise levels; no uncertainty estimate
ARIMA	Linear dynamics for the observed series itself	Rich forecasting theory	No explicit hidden components; needs differencing; awkward for time-varying parameters
State space model	Hidden linear dynamics + noisy observation	Handles trends, drifting coefficients, missing data; gives uncertainty	Needs a model structure and noise parameters; optimality depends on assumptions

The relationships are closer than the table suggests. Every ARIMA model has a state space representation, and the simplest SSM, the local-level model of Section 6.1, is mathematically equivalent to an exponentially weighted moving average, whose smoothing constant is *determined by the noise levels* rather than chosen by hand.

Key takeaways

An SSM = hidden state dynamics + a noisy observation equation. The Kalman filter is the algorithm that solves the filtering problem exactly when everything is linear and Gaussian.

SSMs give not just an estimate but its uncertainty, and update it recursively as each observation arrives.

2. Mathematical Foundations

2.1 The linear Gaussian state space model

The standard model has two equations. The **state transition (process) equation** describes how the hidden state evolves; the **observation (measurement) equation** describes how data are generated from it:

$$x_t = F_t\, x_{t-1} + B_t\, u_t + w_t\,, \quad w_t \sim N(0, Q_t) \tag{1}$$

$$y_t = H_t\, x_t + D_t\, u_t + v_t\,, \quad v_t \sim N(0, R_t) \tag{2}$$

Table 2. Symbols of the linear Gaussian state space model

Symbol	Name	Dimension	Role and financial meaning
x_t	Latent state	n × 1	Hidden quantity: a price level, trend and slope, hedge ratio and intercept. **Never observed.**
y_t	Observation	m × 1	The data we record: a price, a return, the price of asset 1. Often scalar (m = 1).
F_t	Transition matrix	n × n	How the state carries forward. F = 1 gives a random walk; F = [[1,1],[0,1]] adds a drift.
H_t	Observation matrix	m × n	How the state maps to data. In a time-varying regression, H_t contains the regressors (for example $[1, x_t]$).
u_t, B_t, D_t	Control inputs and loadings	—	Optional known external inputs (a known drift, a seasonal dummy). Omitted in most financial examples.
w_t	Process noise	n × 1	Random shocks to the state itself: real changes in the trend or hedge ratio.
v_t	Observation noise	m × 1	Measurement error: microstructure noise, bid–ask bounce, idiosyncratic spread movement.
Q_t	Process covariance	n × n	How fast the hidden state can move. Large Q = state changes quickly.

Symbol	Name	Dimension	Role and financial meaning
R_t	Observation covariance	m × m	How noisy the data are. Large R = observations carry little information.

Assumptions. The noises w_t and v_t are zero-mean, Gaussian, serially uncorrelated and independent of each other and of the initial state $x_0 \sim N(\hat{x}_0, P_0)$. The matrices F_t, H_t, Q_t, R_t are known (or are parameters to be estimated; Chapter 4). Because the model is linear and Gaussian, every conditional distribution of the state is again Gaussian, and so it is fully described by a **mean** (the estimate) and a **covariance** (its uncertainty). That is what makes exact recursive computation possible.

Two different kinds of randomness appear, and keeping them apart is essential. **Process noise** w_t is genuine change in the hidden world: if Q is large, the trend really can move a lot from day to day. **Observation noise** v_t is error in what we see: if R is large, a single price tells us little about the state. The filter's task is to decide, observation by observation, how much of a surprise is *news about the state* and how much is *noise*.

2.2 Why financial observations are noisy proxies

Quoted prices contain microstructure effects (the price bounces between bid and ask), trading-time discreteness, temporary liquidity-driven impact and plain errors. Two assets that are "related" deviate from their relationship by idiosyncratic noise. Returns are noisy measures of expected returns. In each case the thing we care about (value, trend, hedge ratio, expected return) is smoother or slower than the data, and an SSM expresses precisely that: a slowly moving state (small Q) observed through noisy data (larger R).

2.3 A first numerical example: a hidden state generates data

Consider the **local-level model**, where the state is a scalar level that follows a random walk and the data are the level plus noise: $x_t = x_{t-1} + w_t$ and $y_t = x_t + v_t$, with F = H = 1, process standard deviation 0.5 (Q = 0.25) and observation standard deviation 2.0 (R = 4). Starting from $x_0 = 100$, the code below generates 250 days. We, as researchers, know x_t; a practitioner would see only y_t.

Python: simulating a latent state and noisy observations

```
rng = np.random.default_rng(11)
T = 250
w = rng.normal(0, 0.5, T)                  # process noise  (sd 0.5)
v = rng.normal(0, 2.0, T)                  # observation noise (sd 2.0)
x_true = 100 + np.cumsum(w)                # latent state: a random-walk level
y = x_true + v                             # what we actually observe
```

Table 3. First five days of the synthetic series (seed 11)

Day t	Process shock w_t	Hidden state x_t	Obs. noise v_t	Observation y_t
1	0.017	100.017	1.882	101.899
2	0.680	100.697	1.426	102.123
3	0.612	101.309	2.095	103.404
4	-0.255	101.054	-0.778	100.276
5	-0.149	100.905	1.480	102.385

Interpretation. On day 1 the state moves by 0.02 but the analyst sees 101.90, an error of 1.88. Across the sample the state's daily changes have standard deviation 0.46 (true 0.5), but the observations change by 3.04 per day because noise is added each day (and differenced noise has variance 2R). Day-to-day changes in the data therefore *overstate* how fast the underlying level moves (Figure 2). The filter exploits exactly this contrast.

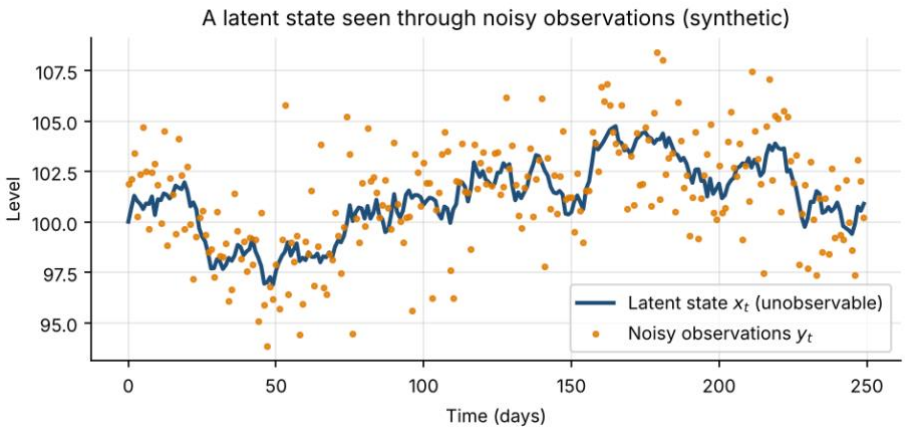


Figure 2. Synthetic local-level data: the hidden random-walk state (blue line) and the noisy observations an analyst would see (orange dots).

Key takeaways

x_t is hidden; $y_t = H_t x_t$ + noise is observed. Q controls how fast the state moves; R controls how noisy the data are.

Only the ratio q = Q/R matters for the filtered state estimate (Section 4.4); the absolute scale matters for uncertainty and the likelihood.

3. The Kalman Filter

3.1 Idea: predict, compare, correct

The **Kalman filter** is a recursive algorithm: it processes observations one at a time, carrying forward only an estimate $\hat{x}_{t|t}$ and its covariance $P_{t|t}$. The subscript "t | s" means *the estimate of the state at time t using data up to time s*. Each step has three moves: **predict** where the state should be now using the dynamics; **compare** the actual observation with the predicted observation (the *innovation*); and **correct** the prediction in proportion to how reliable the new observation is.

Table 4. The six steps of the Kalman filter

Step	Action	Result
1	Initialise $\hat{x}_{0\|0}$ and $P_{0\|0}$	Starting belief and its uncertainty
2	Predict the state	$\hat{x}_{t\|t-1}$, $P_{t\|t-1}$ (uncertainty grows by Q)
3	Predict the observation	$\hat{y}_{t\|t-1} = H_t \hat{x}_{t\|t-1}$
4	Compute the innovation	$e_t = y_t - \hat{y}_{t\|t-1}$ and its variance S_t
5	Update the state estimate	$\hat{x}_{t\|t} = \hat{x}_{t\|t-1} + K_t e_t$
6	Update the uncertainty	$P_{t\|t} = (I - K_t H_t) P_{t\|t-1}$

3.2 The equations

Prediction. Propagate the previous estimate through the transition equation. The uncertainty grows because the state may have been shocked by w_t:

$$\hat{x}_{t|t-1} = F_t \hat{x}_{t-1|t-1} \tag{3}$$

$$P_{t|t-1} = F_t P_{t-1|t-1} F_t^{T} + Q_t \tag{4}$$

Innovation. The innovation e_t is the *forecast error*: the part of the observation that the model did not predict. Its variance S_t combines uncertainty about the state (mapped into observation space by H) and the observation noise:

$$e_t = y_t - H_t \hat{x}_{t|t-1} \tag{5}$$

$$S_t = H_t P_{t|t-1} H_t^T + R_t \tag{6}$$

Kalman gain and update. The gain decides how much of the innovation is passed into the state estimate:

$$K_t = P_{t|t-1} H_t^T S_t^{-1} \tag{7}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t \tag{8}$$

$$P_{t|t} = (I - K_t H_t) P_{t|t-1} \tag{9}$$

Intuition for the gain. In the scalar case with H = 1, $K_t = P_{t|t-1} / (P_{t|t-1} + R)$. If the observation is very noisy (R large relative to P), K ≈ 0 and the filter ignores the new data, trusting its prediction. If the prediction is very uncertain (P large relative to R), K ≈ 1 and the filter largely adopts the new observation. The gain is thus a **precision-weighted average** of prediction and measurement, which is the optimal way to combine two independent Gaussian estimates. The updated uncertainty $P_{t|t} = (1 - K)P_{t|t-1}$ is always smaller than the prediction's: *every observation reduces uncertainty*, even a noisy one. Meanwhile the innovation sequence is the filter's only link to reality; for a correct model the standardised innovations $e_t/\sqrt{S_t}$ are independent N(0, 1), which is the basis for estimation and diagnostics in Chapter 4.

3.3 Worked example: a scalar model, step by step

Take F = H = 1, Q = 1, R = 2, with initial belief $\hat{x}_{0|0} = 10$ and $P_{0|0} = 4$, and observations $y_1 = 12$, $y_2 = 11$, $y_3 = 12.5$. Step 1, by hand:

- **Predict state:** $\hat{x}_{1|0} = 1 \times 10 = 10.0$; $P_{1|0} = 1 \times 4 \times 1 + 1 = 5.0$.
- **Innovation:** $e_1 = 12 - 10 = 2.0$; $S_1 = 5 + 2 = 7.0$.
- **Gain:** $K_1 = 5 / 7 = 0.7143$.
- **Update:** $\hat{x}_{1|1} = 10 + 0.7143 \times 2 = 11.4286$; $P_{1|1} = (1 - 0.7143) \times 5 = 1.4286$.

The filter moved most (71%) of the way from the prediction (10) to the observation (12), because the prediction was uncertain (variance 5) relative to the measurement noise (variance 2). Figure 3 shows the three Gaussian beliefs: the update is narrower than both inputs. The code in Section 7.1 reproduces the full table by machine.

Table 5. Scalar Kalman filter, three steps (output of the code in Section 7.1)

t	y_t	$\hat{x}_{t\|t-1}$	$P_{t\|t-1}$	e_t	S_t	K_t	$\hat{x}_{t\|t}$	$P_{t\|t}$
1	12.0	10.0000	5.0000	2.0000	7.0000	0.7143	11.4286	1.4286
2	11.0	11.4286	2.4286	-0.4286	4.4286	0.5484	11.1935	1.0968
3	12.5	11.1935	2.0968	1.3065	4.0968	0.5118	11.8622	1.0236

Note how the gain falls over time (0.7143 → 0.5484 → 0.5118): as evidence accumulates, uncertainty $P_{t|t}$ shrinks and the filter becomes less willing to be moved by any single observation. It does not fall to zero because Q > 0 keeps re-injecting uncertainty each step, and the gain converges to a **steady-state** value that depends only on Q and R.

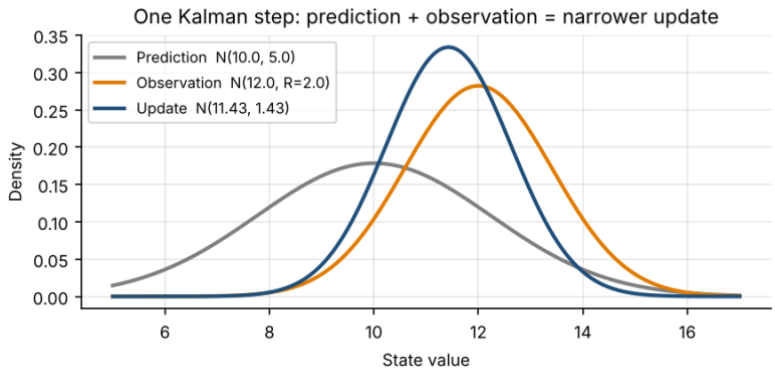


Figure 3. One Kalman step (scalar example, t = 1): the grey prediction and orange observation combine into the narrower blue update.

Python: a general Kalman filter (scalar observation, time-varying H)

```
def kalman_filter(y, F, H, Q, R, x0, P0):
    """Scalar-observation Kalman filter. H has shape (T, n): one row per date."""
    T, n = len(y), len(x0)
    x_p, P_p = np.zeros((T, n)), np.zeros((T, n, n))        # predicted: x_{t|t-1}, P_{t|t-1}
    x_f, P_f = np.zeros((T, n)), np.zeros((T, n, n))        # filtered:  x_{t|t},   P_{t|t}
    e, S, K, ll = np.zeros(T), np.zeros(T), np.zeros((T, n)), np.zeros(T)
    x, P = np.asarray(x0, float), np.asarray(P0, float)
    for t in range(T):
        x_p[t], P_p[t] = F @ x, F @ P @ F.T + Q              # 1. predict the state
        h = H[t]
        e[t] = y[t] - h @ x_p[t]                            # 2. innovation e_t
        S[t] = h @ P_p[t] @ h + R                           #    innovation variance S_t
        K[t] = P_p[t] @ h / S[t]                            # 3. Kalman gain K_t
        x = x_p[t] + K[t] * e[t]                            # 4. update the state
        P = P_p[t] - np.outer(K[t], h) @ P_p[t]             #    (I - K H) P_{t|t-1}
        P = 0.5 * (P + P.T)                                 #    keep P symmetric
        x_f[t], P_f[t] = x, P
        ll[t] = -0.5 * (np.log(2 * np.pi * S[t]) + e[t] ** 2 / S[t])
    return dict(x_p=x_p, P_p=P_p, x_f=x_f, P_f=P_f, e=e, S=S, K=K, ll=ll)
```

3.4 Interpretation on the synthetic series

Running this filter on the 250-day series of Chapter 2 with the true Q = 0.25 and R = 4 gives a root-mean-square error against the hidden state of **0.77**, compared with **2.04** for using the raw observations as the estimate. The filter removes roughly 62% of the error (Figure 4, next chapter). The gain settles at K = 0.2207 and the steady-state prediction variance at 1.1328. These match the closed-form steady-state solution of $P^2 - QP - QR = 0$, namely $P = (Q + \sqrt{(Q^2 + 4QR)})/2 = 1.1328$, giving $K = P/(P + R) = 0.2207$. So each new observation receives a weight of about 22%; this is exactly an exponential moving average with a smoothing constant of 0.22, equivalent to a span of about 8.1 days.

3.5 Assumptions and when to go beyond the standard filter

The Kalman filter is the **minimum-variance estimator among all estimators if the model is linear and Gaussian**, and the best *linear* estimator if the noises are merely uncorrelated with the right covariances. It is *not* a universal solution. It performs well only to the extent that its assumptions hold:

- **Linearity.** Nonlinear transitions or observations (option prices, many volatility models) require the extended or unscented Kalman filter, or particle filters (Chapter 6).
- **Gaussian noise.** Fat-tailed returns make large innovations far more common than the filter expects, so it over-reacts to outliers; robust or heavy-tailed variants are used.
- **Known, correct matrices.** F, H, Q and R must be right or well estimated. A wrong Q or R gives estimates that are still *unbiased in a sense* but sub-optimal, and **reported uncertainties become wrong** (Chapter 4).
- **Independent noises, discrete regimes.** Correlated noise needs an extended state; regime changes call for switching models.

> **Key takeaways**
>
> Predict, compare with the data (innovation), correct with the gain. The gain is a precision-weighted compromise between the model and the newest observation.
>
> The filter is optimal only under linear-Gaussian assumptions; it is a tool, not a guarantee.

4. Smoothing, Forecasting, and Model Estimation

4.1 Filtering, smoothing and forecasting

Table 6. Three questions an SSM answers

Task	Notation	Information used	Use
Filtering	$\hat{x}_{t\|t}$	$y_1, \ldots, y_t$	**Live trading and monitoring.** Best estimate of the state *now*

Task	Notation	Information used	Use
Smoothing	$\hat{x}_{t\|T}$, $T > t$	The whole sample $y_1, \ldots, y_T$	Historical analysis, research, parameter estimation (EM). **Never a live signal**
Forecasting	$\hat{x}_{t+h\|t}$, $\hat{y}_{t+h\|t}$	$y_1, \ldots, y_t$	Predicting future state and observation, with uncertainty

A **filtered** estimate at time t uses only information available at t, so it could have been computed in real time. A **smoothed** estimate revises the past in the light of later data, so it is more accurate but uses the future. Using smoothed estimates in a backtest creates **look-ahead bias**: the backtest sees a hidden state that was only learnable in hindsight. This is the single most important practical distinction in this chapter.

4.2 Rauch–Tung–Striebel smoother

The **RTS smoother** (Rauch, Tung and Striebel, 1965) runs the filter forward, stores the filtered and predicted moments, and then sweeps backward, correcting each filtered estimate with the information in the next smoothed estimate: $\hat{x}_{t|T} = \hat{x}_{t|t} + J_t(\hat{x}_{t+1|T} - \hat{x}_{t+1|t})$, where $J_t = P_{t|t}F^TP_{t+1|t}^{-1}$ is the smoother gain, with a corresponding formula for $P_{t|T}$. At the final date the smoothed and filtered estimates coincide.

Python: RTS smoother

```
def rts_smoother(F, kf):
    """Rauch-Tung-Striebel backward pass: uses ALL observations up to the end of the sample."""
    x_s, P_s = kf["x_f"].copy(), kf["P_f"].copy()
    for t in range(len(x_s) - 2, -1, -1):
        J = kf["P_f"][t] @ F.T @ np.linalg.inv(kf["P_p"][t + 1])       # smoother gain
        x_s[t] = kf["x_f"][t] + J @ (x_s[t + 1] - kf["x_p"][t + 1])
        P_s[t] = kf["P_f"][t] + J @ (P_s[t + 1] - kf["P_p"][t + 1]) @ J.T
    return x_s, P_s
```

Results (Figure 4). On the same series the filtered RMSE is 0.77 and the smoothed RMSE is **0.60**: using the future improves the estimate by about 22%. The posterior standard deviation falls from 0.94 (filtered) to 0.70 (smoothed) mid-sample. The filtered path also visibly **lags** the true state while the smoothed path does not. The final-date estimates are identical (confirmed), as theory requires. The apparent accuracy of the smoother is exactly what a live trader cannot have.

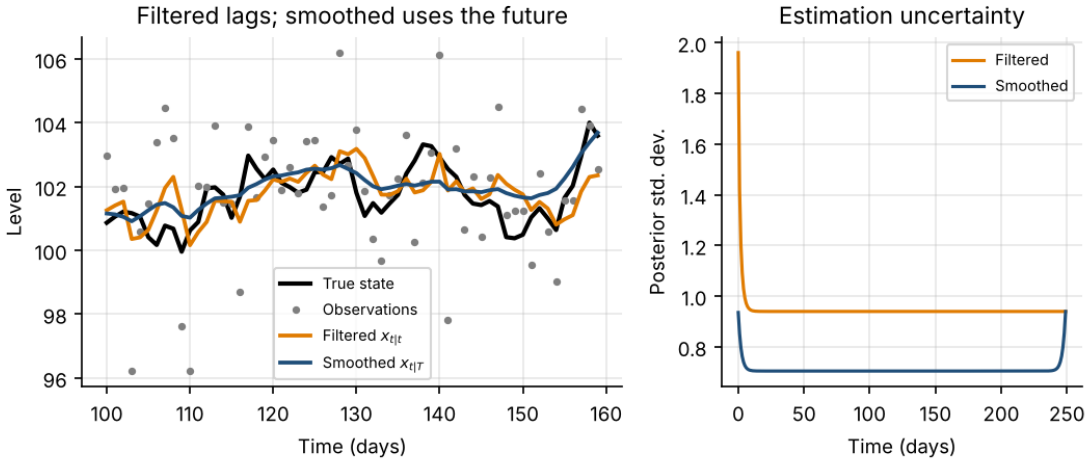


Figure 4. Left: true state, observations, filtered and smoothed estimates (days 100–160). Right: posterior standard deviation; smoothing is more certain because it uses future data.

4.3 Forecasting and the two kinds of uncertainty

The h-step forecast of the state is $\hat{x}_{t+h|t} = F^h\hat{x}_{t|t}$, with variance growing at each step (for the local level, $P_{t+h|t} = P_{t|t} + hQ$). The *observation* forecast adds R. It is vital to separate: **estimation uncertainty** (we do not know where the state is now; $P_{t|t}$), **forecast uncertainty** about the state (it will move further; + hQ) and **observation noise** (the next price will not equal the state; + R). At the end of the synthetic sample the filtered level is 100.54, with the following standard deviations:

Table 7. Uncertainty of the local-level forecast at the last date

Horizon h (days)	State std. dev. √(P + hQ)	Observation std. dev. √(P + hQ + R)
0 (filtered estimate)	0.940	—
1	1.064	2.266
5	1.460	2.476
20	2.425	3.144

Even at h = 1 the *observation* forecast is more than twice as uncertain as the state, because most of what makes tomorrow's price unpredictable is noise, not state movement. Intervals for future prices must use the larger number.

4.4 Initialisation and unknown parameters

The filter needs a starting belief ($\hat{x}_0$, P_0). With little prior knowledge a **diffuse prior** (very large P_0) lets the first observations dominate, as used in the code of this guide; the first one or two innovations are then excluded from the likelihood. A tight but wrong prior is harmful: it takes many steps to unlearn. Initial transients also contaminate early estimates, which is why we skip a burn-in period when evaluating.

In practice Q and R are unknown. Because the innovations e_t are, under the model, independent Gaussians with variances S_t, the **log-likelihood** of the data is available as a by-product of the filter (the *prediction-error decomposition*):

$$\log L(\theta) = -\tfrac{1}{2} \Sigma_t \left[\log(2\pi S_t) + e_t^2 / S_t \right] \tag{10}$$

where θ collects the unknown parameters (Q, R, and possibly entries of F). Maximum likelihood chooses θ to make the innovations as small and as consistent with the predicted variances S_t as possible; it is a numerical optimisation over θ, running the filter at each trial value. Parameters are optimised on a log scale to keep variances positive.

Python: maximum-likelihood estimation through the Kalman filter

```
def fit_mle(y, H, build, theta0, skip=1):
    """Maximum likelihood: the filter's innovations give the Gaussian log-likelihood."""
    def neg_ll(log_theta):
        F, Q, R, x0, P0 = build(np.exp(log_theta))                    # exp() keeps variances positive
        return -kalman_filter(y, F, H, Q, R, x0, P0)["ll"][skip:].sum()
    res = minimize(neg_ll, np.log(theta0), method="Nelder-Mead",
                   options=dict(xatol=1e-4, fatol=1e-6, maxiter=3000))
    return np.exp(res.x), -res.fun
```

Python: fitting the local-level model (parameters: state variance, observation variance)

```
build_level = lambda th: (np.eye(1), np.eye(1) * th[0], th[1], np.array([y[0]]), np.eye(1) * 1e4)
theta_hat, ll_hat = fit_mle(y, H, build_level, theta0=[0.1, 1.0])      # (sigma2_state, sigma2_obs)
```

Applied to the series of Chapter 2 the estimates are σ^2_{state} = **0.1212** and σ^2_{obs} = **4.492** (truth: 0.25 and 4.0), with log-likelihood -561.43. As a correctness check, statsmodels' `UnobservedComponents(level="llevel")` returns 0.1212 and 4.492 with log-likelihood -561.43: the from-scratch filter matches. The state variance is estimated at about half its true value; that is ordinary sampling error from only 250 observations, and the flat top of the log-likelihood in Figure 5 (right) shows why: **Q is weakly identified** in short samples, so parameter uncertainty is substantial.

Expectation–maximisation (EM) is an alternative: it alternates between smoothing (E-step: estimate the hidden states given current parameters) and re-estimating the parameters from the smoothed states (M-step). It is robust to poor starting values and often used to initialise a direct likelihood optimisation. We do not need its derivation here.

4.5 Diagnostics and parameter sensitivity

If the model is right, the **standardised innovations** $z_t = e_t/\sqrt{S_t}$ should be mean zero, variance one, serially uncorrelated and roughly Gaussian. For the fitted model: mean -0.019, standard deviation 1.002, Ljung–Box p-value (10 lags) 0.92 and Jarque–Bera p-value 0.22: no evidence against the model. If Q is set far too small (q = 0.001), the filter is too slow and its errors become persistent: lag-1 innovation autocorrelation rises to 0.22 (versus -0.02) and the Ljung–Box p-value collapses to 2.5e-17 (Figure 5, left).

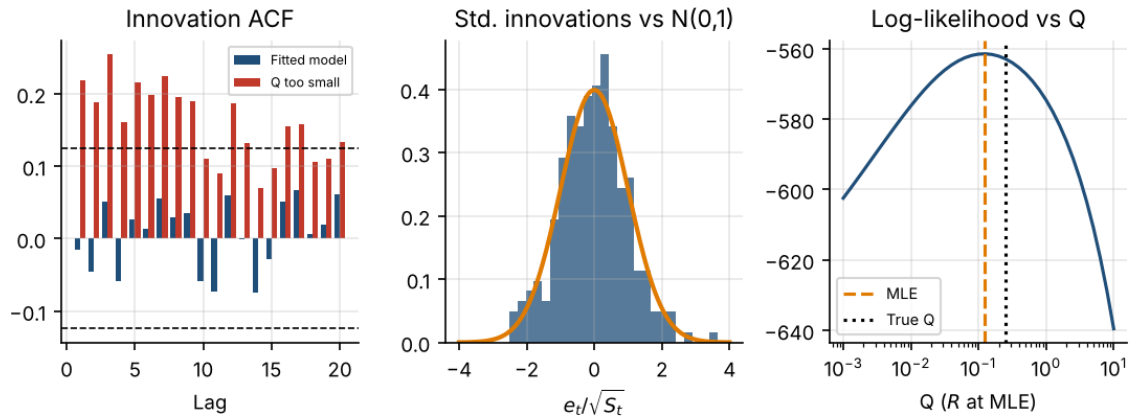


Figure 5. Innovation diagnostics. Left: autocorrelation of standardised innovations (dashed = 95% bounds) for the fitted model and for Q set too small. Centre: histogram versus N(0,1). Right: log-likelihood as Q varies, with the MLE and true values.

Parameter sensitivity matters because Q/R controls the filter's speed. Table 8 holds the data fixed and varies the assumed ratio q = Q/R (R = 4):

Table 8. Effect of the assumed noise ratio q = Q/R on the filtered estimate (true q = 0.0625)

Assumed q	Steady-state gain K	RMSE vs true state	Log-likelihood	Ljung–Box p	Std. of z_t
0.001	0.0311	1.323	-590.9	< 0.001	1.22
0.0625	0.2207	0.767	-563.2	0.66	1.02
4	0.8284	1.682	-663.2	< 0.001	0.58

Interpretation (Figure 6). With q too small the filter barely moves (K = 0.0311) and lags the state; with q too large it chases noise (K = 0.8284) and its error is almost as large as using raw data. The correctly specified ratio is best on RMSE, has the highest likelihood and is the only one whose standardised innovations have standard deviation near one and no autocorrelation. Note also that if Q and R are *both* multiplied by 10 the filtered states are unchanged (maximum difference 0): the state estimate depends only on q, while P, S and the likelihood scale with the level of the noise.

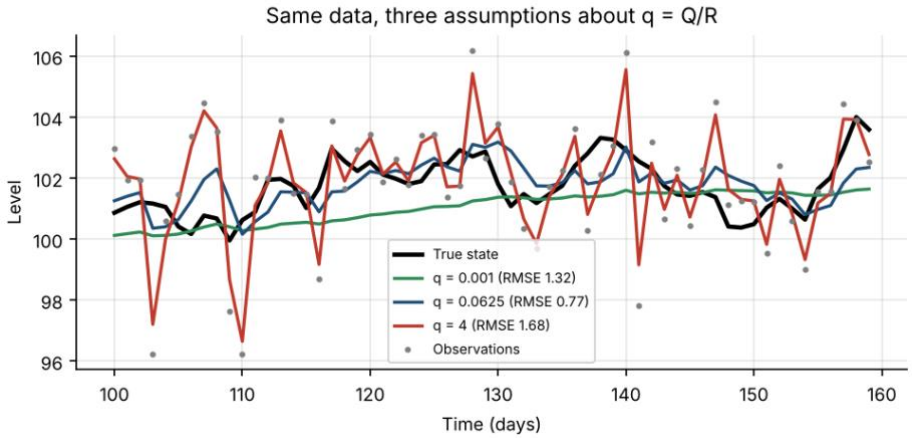


Figure 6. The same data filtered under three assumptions about q = Q/R. Too small a q lags the state; too large a q follows the noise.

Key takeaways

Use filtered estimates for anything that must be implementable in real time; smoothed estimates are for research only.

Fit Q and R by maximum likelihood on training data and **check the standardised innovations**: autocorrelation or variance different from 1 signals misspecification.

Forecast uncertainty = estimation uncertainty + hQ (state movement) + R (observation noise).

5. Financial Applications

5.1 Latent trend estimation

Idea. A price series is a smooth underlying trend plus transitory noise. The **local-linear-trend model** has a hidden level μ_t and hidden slope β_t:

$$\mu_t = \mu_{t-1} + \beta_{t-1} + \xi_t\,, \quad \beta_t = \beta_{t-1} + \zeta_t\,, \quad y_t = \mu_t + \varepsilon_t \tag{11}$$

with F = [[1, 1], [0, 1]], H = [1, 0], Q = diag(σ^2_ξ, σ^2_ζ) and R = σ^2_ε. Unlike a moving average, the filter estimates the **slope** (the drift) as well as the level, and its speed adapts to the signal-to-noise ratio. A 400-day synthetic price (seed 5) was generated with true variances (level 0.09, slope 0.0001, noise 2.25); maximum likelihood recovers (0.105, 0.00024, 2.08).

Result (Figure 7). Excluding the first 50 days, the Kalman level has RMSE **0.65** against the true level, versus **1.50** for a 20-day moving average and 1.44 for the raw price. The moving average is *no better than the raw price* here because it lags a falling price: its mean error is 1.06 versus 0.17 for the filter. The filtered slope tracks the true drift with RMSE 0.069 per day.

When appropriate: the series has a persistent component and transitory noise, and the signal is meant to be directional (trend following, drift estimation). **Failure modes:** jumps and regime shifts violate the smooth-trend assumption; over-fitting Q makes the "trend" follow noise; and a trend that is estimated is not necessarily a trend that persists.

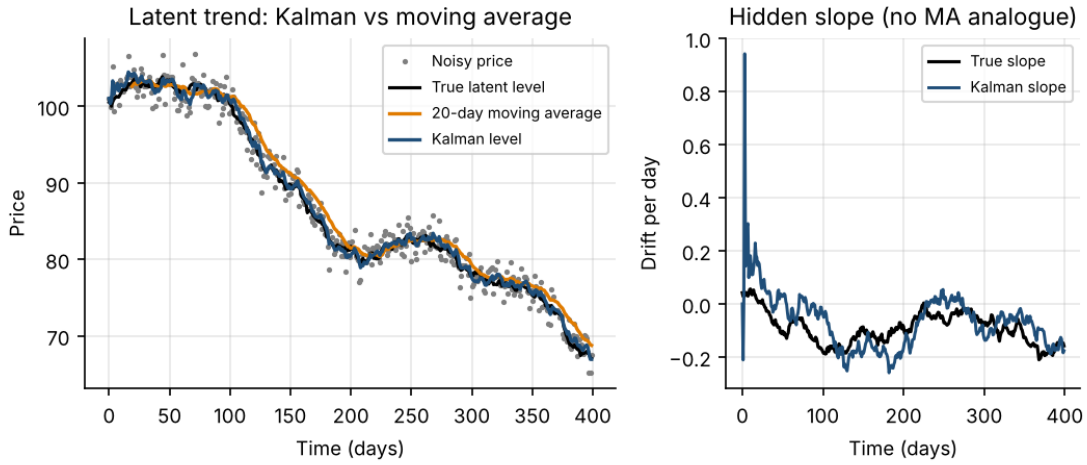


Figure 7. Local-linear-trend model on synthetic prices. Left: Kalman level versus a 20-day moving average (which lags). Right: the hidden slope and its filtered estimate (early values reflect initialisation).

5.2 Time-varying regression and dynamic hedge ratios

Idea. In $y_t = \alpha_t + \beta_t x_t + \varepsilon_t$ let the coefficients drift as random walks, so the state is $\theta_t = [\alpha_t, \beta_t]^T$, F = I, and $\mathbf{H_t = [1, x_t]}$: the observation matrix contains the regressor and therefore *changes every day*. The filter is then a recursive least-squares regression with exponential forgetting whose speed is governed by Q. This is the standard model for hedge ratios and time-varying factor exposures (betas to market, value, momentum). **Appropriate when** exposures genuinely drift slowly; **fails when** relationships change abruptly (breaks), when regressors are highly collinear (α and β poorly separated; centring x helps), or when Q is chosen to chase noise.

5.3 Pairs trading

Idea. If two assets share a long-run relationship (cointegration; Engle and Granger, 1987), the spread $y_t - \beta_t x_t - \alpha$ is mean-reverting and deviations can be traded: buy the cheap leg, sell the expensive leg. The dynamic hedge ratio from Section 5.2 makes the spread robust to slowly moving β. A natural trading signal is the **standardised innovation** $z_t = e_t/\sqrt{S_t}$: it measures how far today's price is from the filter's prediction in units of its forecast standard deviation (Chan, 2013). **Appropriate when** the pair has an economic reason to be linked and the spread has historically reverted; **fails**

when the link breaks (merger, regulatory change, a structural shift), when costs exceed the reversion profit, or when the filter *adapts to the divergence* and absorbs it into β, erasing the signal (see Chapter 9).

5.4 Market regimes

Equity markets alternate between calm and turbulent periods, expansions and recessions. A **linear Gaussian SSM has continuous, smoothly evolving states**; it cannot represent a state that jumps between a few discrete regimes. Regimes are modelled by **hidden Markov models (HMM)** and **Markov-switching models** (Hamilton, 1989), where a discrete hidden state $S_t \in \{1, \ldots, K\}$ follows a Markov chain and the parameters of the observation distribution depend on S_t. They are state space models in a broad sense but use different inference (the Hamilton filter, a discrete analogue of the Kalman filter) and are *not* interchangeable with the linear Gaussian case. Regime probabilities are useful for scaling risk, but regimes are identified with delay and estimated probabilities can flip-flop.

5.5 Volatility estimation

Volatility is a latent state: only returns r_t are seen. In the **stochastic volatility** model $r_t = \exp(h_t/2)\,\varepsilon_t$, where the log-variance h_t follows an AR(1) process. The transition for h_t is linear, but the observation is *nonlinear* in the state and the noise is multiplicative. Taking logs gives $\log r_t^2 = h_t + \log \varepsilon_t^2$, a linear observation equation whose error $\log \varepsilon_t^2$ is **far from Gaussian** (a skewed log-χ^2 distribution). Applying the standard Kalman filter to this transformed model is the **quasi-maximum-likelihood** method of Harvey, Ruiz and Shephard (1994): it is simple but approximate and inefficient. Exact treatment needs particle filters or MCMC. So a standard linear Gaussian Kalman filter does **not** directly solve stochastic-volatility estimation, and GARCH-type models, which make volatility a deterministic function of past returns, are a common practical alternative.

5.6 Signal extraction

More generally an SSM separates a **persistent component** (a slowly moving signal such as a value spread, a carry signal, or the efficient price) from a **transitory component** (noise or short-lived dislocations). The local-level model does this with one parameter, $q = Q/R$; stacking components (trend + cycle + seasonal) gives *structural time series* models (Harvey, 1989). The assumption that does the work is that the persistent part is smoother than the noise: if the "noise" is actually informative or persistent, the filter throws away signal.

Table 9. Applications: when each is appropriate and how it fails

Application	Latent state	Appropriate when	Main failure mode
Trend estimation	Level (and slope)	Persistent drift plus transitory noise	Jumps; Q tuned to noise; trend does not persist
Dynamic hedge ratio	Coefficients [α, β]	Slowly drifting exposures	Abrupt breaks; collinearity; Q too large or small
Pairs trading	β and spread	Economically linked, mean-reverting pair	Link breaks; costs; filter absorbs divergence
Regimes	Discrete regime S_t	Distinct, recurring market states	Late detection; unstable probabilities; wrong K
Volatility	Log-variance h_t	Moderate frequency, QML acceptable	Non-Gaussian log χ^2 error; nonlinearity
Signal extraction	Persistent component	Signal smoother than noise	Noise is informative; wrong component split

Key takeaways

The same machinery covers trends, drifting regressions and spreads; what changes is the design of F, H and Q.

Regime models and stochastic volatility need discrete or nonlinear / non-Gaussian extensions; do not force them into the linear Gaussian template.

6. Important Model Variants

Local-level model. $x_t = x_{t-1} + w_t$, $y_t = x_t + v_t$. The filtered estimate is an exponentially weighted moving average whose weight K is set by q = Q/R. **Local-linear-trend model.** Adds a stochastic slope (Section 5.1); equivalent to double exponential smoothing.

Dynamic linear regression / time-varying parameter (TVP) models. Regression coefficients form the state and H_t carries the regressors (Section 5.2). TVP models also include richer coefficient dynamics (AR(1) coefficients, mean-reverting betas) and often *discount factors* instead of an explicit Q.

General linear Gaussian SSM. The full model of Chapter 2: any combination of components (trend, cycle, seasonal, regression, ARMA errors) can be written with the appropriate F and H, and all of the filtering, smoothing and likelihood machinery applies unchanged (Durbin and Koopman, 2012).

Hidden Markov and Markov-switching models. The latent state is discrete; inference is by the Hamilton filter or forward–backward algorithm. Used for regimes. They are *related* to Kalman filtering (both are recursive Bayesian filters) but are not special cases of the linear Gaussian SSM. **Extended (EKF) and unscented (UKF) Kalman filters** handle nonlinear F or H: the EKF linearises with Jacobians; the UKF propagates a small set of sigma points (Julier and Uhlmann, 2004) and avoids derivatives. **Particle filters** (Gordon, Salmond and Smith, 1993) represent the state distribution by weighted random samples and handle any dynamics and noise, at much higher computational cost.

Table 10. Comparison of state space model variants

Model	Purpose	Key assumptions	Advantages	Limitations
Local level	Smooth a series; extract persistent level	Random-walk level; Gaussian noise	One parameter; equals an adaptive EWMA	No trend; lags a drifting series
Local linear trend	Level and drift estimation	Random-walk level and slope	Estimates slope with uncertainty	Slope noisy; breaks hurt; 3 parameters
Dynamic linear regression	Time-varying hedge ratios and betas	Coefficients follow random walks; H_t known	Recursive, adapts, gives coefficient uncertainty	Choice of Q critical; collinearity; no abrupt breaks
TVP models	Evolving relationships and exposures	Coefficient dynamics specified	Flexible dynamics; discount-factor variants	Overfitting; many parameters
Linear Gaussian SSM	General framework	Linear, Gaussian, known/estimated matrices	Exact filtering, smoothing, likelihood	Fails for nonlinearity, fat tails, regimes
HMM / Markov switching	Discrete regimes	Finite hidden states; Markov transitions	Regime probabilities; captures sudden shifts	Delayed detection; K and label choice; not a Kalman filter
EKF / UKF	Nonlinear dynamics or observations	Approximately Gaussian posterior	Fast; reuse Kalman structure	EKF linearisation error; UKF no guarantee; poor for multimodal
Particle filter	General nonlinear / non-Gaussian	Any model; enough particles	Few distributional restrictions	Costly; degeneracy; hard to tune

Key takeaways

Local-level and local-linear-trend are the workhorse special cases; dynamic regression is the finance workhorse.

HMMs and nonlinear filters answer different questions from the linear Gaussian Kalman filter; choose by the structure of the problem, not by name.

7. Practical Python Implementations

All implementations use the general `kalman_filter` of Section 3.3 (printed once) and `fit_mle` of Section 4.4, with NumPy and SciPy. We implement the filter from scratch because it makes every step visible and verifiable; production code would normally use `statsmodels.tsa.statespace` (which provides the same filter, smoother and likelihood, plus diagnostics) or FilterPy for nonlinear filters. FilterPy was not available in the environment used here and was not used. All data are synthetic.

7.1 A scalar Kalman filter

Python: the Section 3.3 worked example

```
y_demo = np.array([12.0, 11.0, 12.5])
demo = kalman_filter(y_demo, np.eye(1), np.ones((3, 1)), Q=np.eye(1) * 1.0, R=2.0,
                     x0=np.array([10.0]), P0=np.eye(1) * 4.0)
```

The output (Table 5) agrees with the hand calculation. The only choices are the model matrices (here 1 × 1), the noise variances Q and R, and the initial belief ($\hat{x}_0$, P_0); the outputs are the filtered states and covariances plus the innovations e and S used for the likelihood and diagnostics. On the 250-day series the same function reproduces the results of Chapters 3–4 (filtered RMSE 0.77) and matches statsmodels' likelihood.

7.2 Estimating a latent trend

Python: local-linear-trend model fitted by maximum likelihood

```
F2 = np.array([[1.0, 1.0], [0.0, 1.0]]); H2 = np.column_stack([np.ones(T2), np.zeros(T2)])
b2 = lambda th: (F2, np.diag([th[0], th[1]]), th[2], np.array([y2[0], 0.0]), np.diag([1e4, 1.0]))
th2, ll2 = fit_mle(y2, H2, b2, [0.1, 0.001, 2.0], skip=2)
kf2 = kalman_filter(y2, F2, H2, np.diag(th2[:2]), th2[2], np.array([y2[0], 0.0]), np.diag([1e4, 1.0]))
```

The state is [level, slope]; the code specifies F, H, Q, R, estimates the three variances by maximum likelihood (the diffuse prior on the level and a unit prior on the slope) and filters. The first two innovations are skipped in the likelihood. Results are in Section 5.1: RMSE 0.65 for the filter versus 1.50 for the moving average. The key parameter is $q_{slope} = \sigma^2_\zeta/\sigma^2_\varepsilon$: increasing it makes the slope estimate more responsive but noisier.

7.3 A time-varying hedge ratio

We generate a synthetic pair of log prices: x_t follows a random walk, the true β_t drifts as a random walk, and the spread s_t is mean-reverting (an AR(1) with coefficient 0.93), so that $y_t = 0.3 + \beta_t(x_t - x_0) + s_t$. The regressor is centred at its starting value to reduce collinearity with the intercept. The state is $[\alpha_t, \beta_t]$; the intercept is held almost constant (variance 10^{-9}) while β may drift.

Python: synthetic pair generator (shared with Chapter 8)

```
def make_pair(seed, T=1500, brk=None):
    """Synthetic log prices: y_t = alpha + beta_t * (x_t - x_0) + s_t, beta a random walk, s_t a mean-reverting
spread."""
    rng = np.random.default_rng(seed)
    x = 4.6 + np.cumsum(rng.normal(0, 0.015, T))
    beta = 1.0 + np.cumsum(rng.normal(0, 0.003, T))
    phi, sd = np.full(T, 0.93), np.full(T, 0.008)
    if brk is not None:
        beta[brk:] += 0.10                      # relationship shifts
        phi[brk:] = 1.0                         # spread stops mean-reverting
    s = np.zeros(T)
    for t in range(1, T): s[t] = phi[t] * s[t - 1] + rng.normal(0, sd[t])
    return x, 0.3 + beta * (x - x[0]) + s, beta
```

Python: dynamic hedge ratio by Kalman filter

```
def build_hedge(th):                            # th = (q_beta, r); alpha is (almost) constant
    return np.eye(2), np.diag([1e-9, th[0]]), th[1], np.array([0.0, 1.0]), np.diag([10.0, 1.0])


def run_hedge(x, y, n_train, theta=None, q_beta=1e-5):
    Hp = np.column_stack([np.ones(len(x)), x - x[0]])   # y_t = alpha_t + beta_t * (x_t - x_0) + noise
    if theta is None:                           # q_beta is a researcher's prior; R is fitted on the TRAINING
window only
        r_hat, _ = fit_mle(y[:n_train], Hp[:n_train], lambda th: build_hedge((q_beta, th[0])), theta0=[5e-4])
        theta = np.array([q_beta, r_hat[0]])
```

```
    F_, Q_, R_, x0_, P0_ = build_hedge(theta)
    return kalman_filter(y, F_, Hp, Q_, R_, x0_, P0_), theta   # then filter chronologically
```

A modelling choice that matters. The variance q_β is set to 10^{-5} as a researcher's prior (a hedge ratio moving about 0.3% per day), and only the observation variance R is estimated on the first 750 days ($R = 2.17 \times 10^{-4}$). Section 9 and Chapter 8 report what happens if both are left to maximum likelihood. Because the data-generating β moves with variance 9×10^{-6}, this prior is close to the truth; in practice that closeness is never known, so we test ×0.1 and ×10 in Chapter 8.

Table 11. Hedge-ratio estimation error, days 150–1500 (true β known by construction)

Method	RMSE vs true β	Std. dev. of daily change in estimate
Kalman filter (filtered, real-time)	0.0259	0.0030
90-day rolling OLS	0.1355	0.0105
Static OLS (first 750 days, β = 0.938)	0.0256	0

Interpretation (Figure 8). The filter is five times more accurate than the 90-day rolling regression, whose estimates are volatile because each window contains little variation in x_t. But in this sample the true β drifts only from 1.00 to 0.95, so a **static** estimate from the training window does about as well as the filter (0.026 versus 0.026). A time-varying model earns its keep only when the relationship genuinely moves; here the main benefit is hedging against drift that is not known in advance. The large early errors are the initialisation transient.

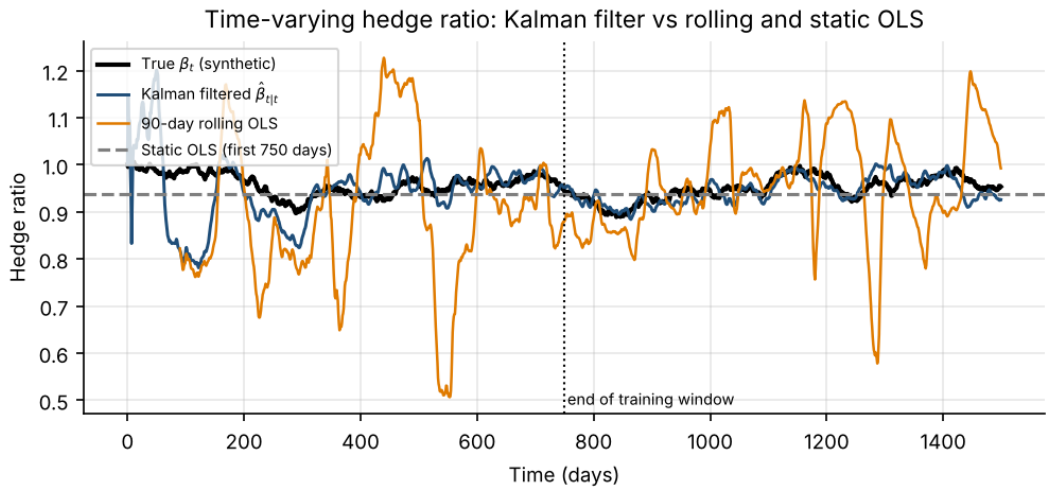


Figure 8. Estimated hedge ratio: Kalman filter, 90-day rolling OLS and a static OLS estimate from the training window, against the true synthetic β.

7.4 A basic pairs-trading research example

Signal. At each date the innovation $e_t = y_t - (\hat{\alpha}_{t|t-1} + \hat{\beta}_{t|t-1}(x_t - x_0))$ is the surprise in y relative to its prediction from the hedge-ratio model; dividing by $\sqrt{S_t}$ **standardises** it so a threshold is meaningful across periods. A positive z_t means y is expensive relative to x. **Rules:** enter short the spread (short y, long $\hat{\beta}$ units of x) when $z_t > 1.5$, enter long when $z_t < -1.5$; exit when $|z_t| < 0.3$. The thresholds were fixed in advance and were not optimised. **No look-ahead:** the position decided at the close of day t earns the return from t to t + 1 and uses the hedge ratio $\hat{\beta}_{t|t}$ known at t.

Python: signal, positions and a simple cost-aware backtest

```
def positions(z, entry=1.5, exit=0.3, stop=None):
    pos, out, blocked = 0, np.zeros(len(z)), False
    for t, zt in enumerate(z):
        if stop is not None and abs(zt) > stop: pos, blocked = 0, True      # stop-out; stay flat
        elif blocked and abs(zt) < exit: blocked = False
        elif not blocked:
            if pos == 0: pos = 1 if zt < -entry else (-1 if zt > entry else 0)
            elif (pos == 1 and zt > -exit) or (pos == -1 and zt < exit): pos = 0
        out[t] = pos
    return out

def backtest(y, x, beta_f, z, cost=0.0005, delay=0, **kw):
```

```
    pos = positions(z, **kw)
    if delay: pos = np.r_[np.zeros(delay), pos[:-delay]]                    # trade one bar later
    gross = np.r_[0.0, pos[:-1] * (np.diff(y) - beta_f[:-1] * np.diff(x))]  # position set at t earns t -> t+1
    trade = np.abs(np.diff(np.r_[0.0, pos])) * (1 + np.abs(beta_f))         # notional traded at t
    return gross, gross - cost * trade, pos
```

Execution and costs. Costs are 5 basis points per unit of notional traded (both legs; hedge-ratio rebalancing is ignored), trades occur at the same close that generates the signal, and capital is one unit of notional in leg y. Those assumptions are favourable; Table 12 relaxes some of them on the 750-day test period (days 750–1499) of a stable synthetic pair.

Table 12. Research backtest, stable synthetic pair, days 750–1499 (illustrative only)

Variant	Ann. return	Ann. vol.	Sharpe	Max drawdown	Entries
Gross of costs	12.2%	5.0%	2.45	2.9%	25
Net of 5 bp costs	10.5%	4.8%	2.19	3.0%	25
Net 5 bp, trade one day late	7.7%	4.8%	1.61	3.0%	—
Net 10 bp costs	8.9%	4.7%	1.89	3.2%	—
Net 20 bp costs	5.6%	4.6%	1.23	4.0%	—

How to read this. The spread in this synthetic world was *designed* to mean-revert, so a positive result is built in; it says the code works, not that pairs trading pays. Even here the Sharpe ratio falls from 2.45 gross to 2.19 net of costs, to 1.61 if execution is one day late and to 1.23 at 20 bp. With about three years of data the standard error of an annualised Sharpe ratio is roughly 0.6, so differences of this size are not statistically meaningful. The strategy makes about 17 position changes (entries and exits) a year while invested about 15% of the time. **Real-world issues** omitted here: borrow and financing costs, short-sale constraints, market impact and capacity, bid–ask spreads on both legs, and the cost of rebalancing β. **Risk controls** worth building: a stop on $|z_t|$, a maximum holding period (spreads that have not reverted by then are probably broken), position limits, and the model-health monitor of Chapter 8. A backtest of this kind is a research tool, not proof of profitability (Bailey et al., 2014).

Key takeaways

Use the standardised innovation as the signal and keep the timing strict: information at t, position at t, return from t to t + 1.

A hedge-ratio filter beats a short rolling regression for stability, but beats a static estimate only if the relationship really drifts.

Report results gross and net, with delayed execution and higher costs, and with standard errors.

8. Integrated Quantitative Trading Case Study

This chapter builds an end-to-end research pipeline on a **second synthetic pair** (seed 21) that includes a **structural break** on day 1100: the true hedge ratio jumps by +0.10 and the spread stops mean-reverting (its AR coefficient goes from 0.93 to 1.0, a random walk). The point is not to make money but to show the full workflow, with honest out-of-sample evaluation, and to see what failure looks like.

Table 13. The ten steps of the case study

#	Step	Specification
1	Objective	Estimate a time-varying hedge ratio between assets Y and X; trade the standardised spread deviation; detect when the relationship breaks
2	Observables and latent states	Observed: log prices y_t, x_t. Latent: intercept α_t and hedge ratio β_t
3	Equations	State: $[\alpha_t, \beta_t]^T = [\alpha_{t-1}, \beta_{t-1}]^T + w_t$. Observation: $y_t = \alpha_t + \beta_t(x_t - x_0) + v_t$
4	Parameters	$q_\beta = 10^{-5}$ (prior), $q_\alpha = 10^{-9}$; R fitted by maximum likelihood on days 0–749 only: $R = 2.57 \times 10^{-4}$

#	Step	Specification
5	Chronological filtering	Filter days 0–1499 one at a time; parameters frozen after day 749 (no refitting)
6	Signal	$z_t = e_t/\sqrt{S_t}$; enter at $\|z\| > 1.5$, exit at $\|z\| < 0.3$; optional stop at $\|z\| > 3$
7	Stability / usefulness	Hedge-ratio error, innovation diagnostics, P&L by regime
8	Out-of-sample test	Train days 0–749; test days 750–1499 split at the break into 750–1099 (pre) and 1100–1499 (post)
9	Costs and risk	5 bp per unit notional; sensitivity to cost, delay and Q; stop-out; health monitor
10	Failure conditions	Identified in Section 8.4

8.1 Model fit and signal

Figure 9 shows the fitted pair. Panel (a) plots observed log prices against the filter's one-step-ahead predictions: the filter tracks the series closely, and the gap between the two lines *is* the innovation. Panel (b) shows the filtered hedge ratio against the truth: it follows the slowly drifting β through the pre-break test period (RMSE 0.030) but after day 1100 it **overshoots badly** (RMSE 0.272, with the estimate peaking near 1.9 against a true value of about 1.4) because the spread no longer mean-reverts and the filter misattributes the drifting spread to the hedge ratio. Panel (c) shows the estimated spread; panel (d) the signal z_t with entry and exit bands. The signal is visibly more extreme after the break.

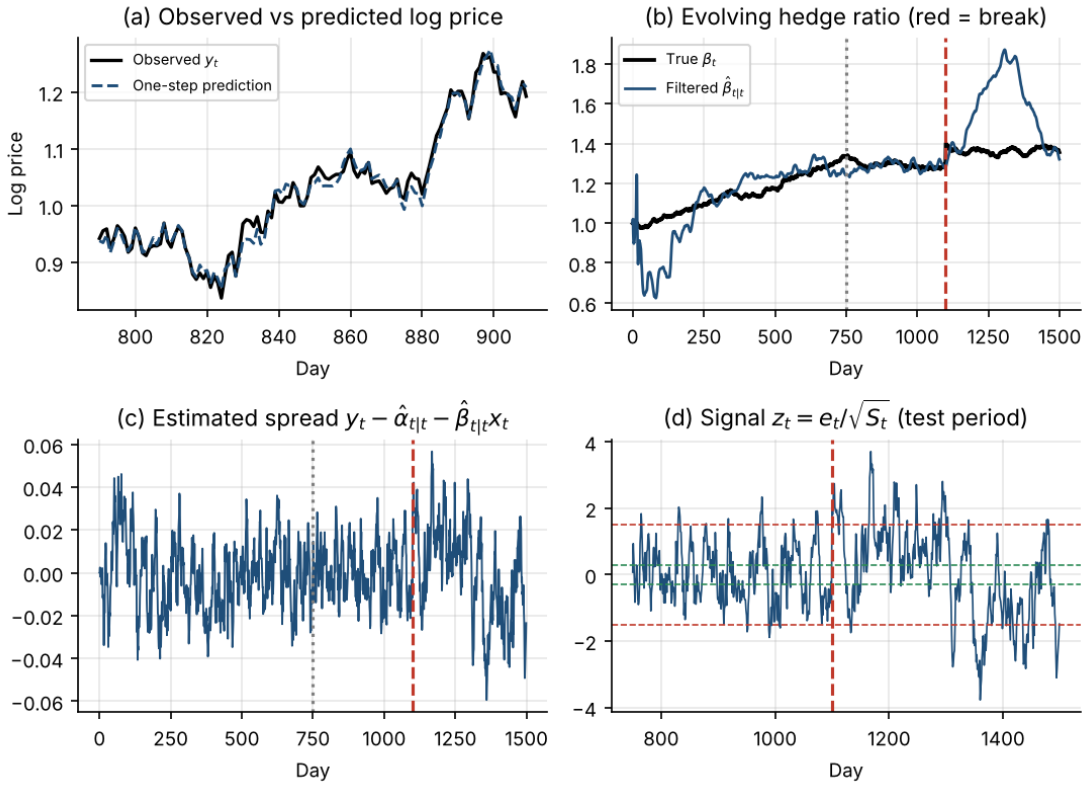


Figure 9. Case study. (a) Observed vs predicted log price; (b) hedge ratio, filtered vs true; (c) estimated spread; (d) trading signal with entry (red) and exit (green) bands. Dotted line = end of training; dashed red = structural break.

8.2 Evaluation: stability and economic usefulness

Table 14. Out-of-sample results by regime (5 bp costs unless stated)

Metric	Test, pre-break (350 d)	Test, post-break (400 d)
Hedge-ratio RMSE	0.030	0.272
Std. dev. of z_t (expected ≈ 1 for a well-specified model)	0.86	1.45
Lag-1 autocorrelation of z_t	0.81	0.93

Metric	Test, pre-break (350 d)	Test, post-break (400 d)
Share of days with $\|z_t\| > 1.5$	8%	31%
Gross Sharpe ratio	1.87	0.57
Net Sharpe ratio	1.64	0.33
Net annualised return	9.0%	3.4%
Maximum drawdown (log-points)	4.1%	10.5%
Entries / share of time invested	10 / 18%	16 / 59%
Net Sharpe with $\|z\| > 3$ stop	1.64	0.22

Interpretation. Before the break the strategy earns a net Sharpe of 1.64 with a maximum drawdown of 4.1%. After the break the *diagnostics* change sharply (hedge-ratio error 9×, signal volatility 1.45 versus 0.86, signals beyond ±1.5 on 31% of days instead of 8%), the strategy is invested 59% of the time instead of 18% and drawdown is 2.5× larger. Yet the post-break Sharpe ratio is still positive (0.33). **P&L is an unreliable break detector**: with 400 days the standard error of the Sharpe ratio is about 0.8, so the drop from 1.64 to 0.33 cannot be separated from luck, whereas the innovation statistics reveal the break clearly. The stop-out rule ($|z| > 3$) cut time invested and drawdown (9.2% versus 10.5%) but did not improve the Sharpe ratio. The innovations are strongly autocorrelated even before the break (lag-1 0.81) because the white-noise observation error cannot represent a persistent spread: the model is *misspecified* from the start, which is common in practice and is why the z-score's "standard deviation" need not be exactly 1.

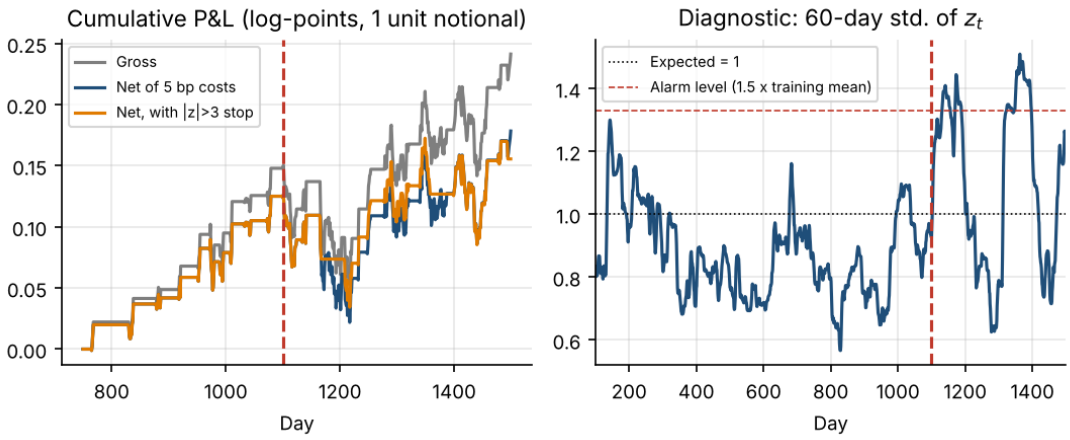


Figure 10. Left: cumulative test-period P&L (log-points per unit of notional) gross, net of 5 bp costs and net with a $|z| > 3$ stop. Right: 60-day standard deviation of z_t with an alarm level fixed from training data.

8.3 Sensitivity, costs and model monitoring

Table 15. Sensitivity of the pre-break test period (days 750–1099)

Variation	Net Sharpe	Entries	Hedge-ratio RMSE
$q_\beta \times 0.1$	2.27	16	0.0287
$q_\beta \times 1$ (base)	1.64	10	0.0304
$q_\beta \times 10$	1.32	1	0.0349
Cost 0 bp per unit traded	1.87	—	—
Cost 5 bp per unit traded	1.64	—	—
Cost 10 bp per unit traded	1.38	—	—
Cost 20 bp per unit traded	0.79	—	—
Execution one day later (5 bp)	1.22	—	—

Larger costs reduce the Sharpe ratio steadily (1.87 gross to 0.79 at 20 bp) and one-day-late execution costs about 0.41 units at the base cost. Results are far more stable across hedge-ratio RMSE than across trading outcomes: ×0.1 and ×1 give similar estimation accuracy but different trade counts. **Do not pick the best row**: choosing q_β to maximise out-of-sample Sharpe contaminates the test set. A better approach is to choose parameters on training data (or by walk-forward validation) and *report* the spread of outcomes across reasonable values. At ×10 the filter absorbs nearly every deviation and the strategy takes 1 trade, so its Sharpe ratio is meaningless.

Model-health monitor. A rule fixed *before* looking at the test data: raise an alarm if the 60-day standard deviation of z_t exceeds 1.5× its mean over days 150–749 (threshold 1.33). It produced 0 false alarms in the pre-break test window and first fired on day 1131, **31 days after the break** (right panel of Figure 10). That delay is the price of using 60 days of evidence; a shorter window reacts faster but raises false alarms.

8.4 When this model fails

- **Structural break in the relationship** (hedge ratio jump, spread no longer stationary): the filter chases the divergence, the signal misleads and losses compound while the position stays open.
- **Misspecified observation noise.** A persistent spread is treated as white noise; the likelihood then prefers an over-flexible β. On this pair, letting maximum likelihood choose *both* q_β and R on the training data gave q_β = 1.5e-3 (150 times the prior), a signal with standard deviation 0.34 and **0 trades** in the pre-break test: the filter absorbed the spread into β and the signal vanished. Statistical fit and trading usefulness are different objectives.
- **Regime-dependent costs and liquidity**, borrow constraints and crowding, none of which are in the model.
- **Short data and parameter uncertainty:** with one test path per regime, differences in Sharpe ratio are mostly noise.

> **Key takeaways**
>
> Judge a state space strategy by innovation diagnostics and parameter stability, not just P&L.
>
> Freeze parameters at the end of training, filter chronologically, and fix monitoring rules before looking at test data.
>
> Maximum likelihood optimises one-step prediction, not trading profit; a filter that predicts well may trade badly.

9. Limitations and Common Mistakes

Table 16. Common mistakes, their symptoms and diagnostics

Mistake	Symptom	Diagnostic or remedy
Wrong transition or observation equations	Systematic innovation patterns; poor forecasts	Plot innovations; try richer F, H; compare AIC/BIC across structures
Latent states not identified	Flat likelihood; unstable estimates; α/β swap roles	Profile the likelihood; centre regressors; fix or constrain some parameters
Miscalibrated Q, R	Estimate too lagged or too noisy; standardised innovations not unit variance	Fit by ML on training data; check Table 8-style sensitivity; Ljung–Box and variance of z_t
Over-fitting, unstable parameters	Parameters change sharply between samples; great in-sample, poor out-of-sample	Walk-forward validation; parsimony; report ranges across Q
Non-stationarity, structural breaks	Innovations drift in size and sign; hedge ratio jumps	Rolling diagnostics; health monitor (Section 8.3); re-estimation rules
Non-Gaussian, heavy-tailed returns	Occasional huge innovations; over-reaction; coverage too low	Check kurtosis of z_t; robust or Student-t filters; winsorise innovations
Correlated noise, misspecified covariance	Autocorrelated innovations (as in Section 8.2)	Model the correlated component as part of the state (e.g. AR(1) spread)

Mistake	Symptom	Diagnostic or remedy
Confusing filtered and smoothed	Backtest too good; vanishes live	Only $\hat{x}_{t\|t}$ in signals; audit the code path
Look-ahead bias and leakage	Test results better than any realistic expectation	Fit parameters on training window only; strict t / t+1 timing; no refit using test data
Statistical fit mistaken for profit	Good likelihood, no profit (Section 8.4)	Evaluate economic metrics net of costs, with delays
Ignoring costs and turnover	Gross edge eaten by costs	Cost sensitivity; trade only when the expected reversion exceeds costs

9.1 Model risk versus estimation uncertainty

Estimation uncertainty is the filter's *own* statement of how well it knows the state given the model (the covariance $P_{t|t}$) plus the uncertainty in estimated parameters (visible in the flat likelihood of Figure 5). It can in principle be reduced with more data. **Model risk** is the risk that the model structure is wrong: a break occurred, noise is fat-tailed, the spread is persistent. The filter's $P_{t|t}$ says nothing about it; it is computed *assuming the model is right*. After the break in Chapter 8 the filter's reported uncertainty gave no warning, while the innovation diagnostics did. This is why monitoring standardised innovations matters: they are the only direct test of the model against reality.

Common mistakes

Treating $P_{t|t}$ as the total uncertainty. It is conditional on the model and parameters being correct.

Re-fitting Q and R on the test period, or choosing thresholds and priors by out-of-sample Sharpe.

Reporting a smoothed hedge ratio in a backtest because it "looks cleaner".

10. Interview Revision and Formula Sheet

10.1 Ten interview questions

Q1. What is the difference between a state space model and a Kalman filter?

A. The state space model is the *model*: a transition equation for the hidden state and an observation equation linking it to data. The Kalman filter is an *algorithm* that recursively computes the conditional mean and covariance of the state given the data, and is optimal when the model is linear-Gaussian. The same SSM can be handled by other algorithms (smoother, particle filter), and the Kalman filter can be used as an approximation outside its assumptions.

Q2. What determines the Kalman gain?

A. The relative sizes of the prediction uncertainty $P_{t|t-1}$ and the observation noise R: $K = P_{t|t-1}H^T(HP_{t|t-1}H^T + R)^{-1}$. In the scalar case $K = P/(P + R)$. Large P (or small R) gives K near 1 (trust the data); small P (or large R) gives K near 0 (trust the model). P itself depends on Q, so the gain depends on Q/R and, over time, converges to a steady state.

Q3. How do process noise and observation noise affect the estimate?

A. Higher Q means the state is believed to move more, so predictions are less certain, the gain rises and the estimate reacts faster but is noisier. Higher R means data are less reliable, so the gain falls and the estimate is smoother but lags. Only the ratio Q/R affects the filtered state, while the levels of Q and R scale the reported uncertainty (Table 8).

Q4. What is the difference between filtering and smoothing?

A. Filtering estimates x_t using data up to t; smoothing estimates it using data up to a later time $T > t$, so it is more accurate but is not available in real time. Using smoothed states in a backtest is look-ahead bias. The two coincide at the final date.

Q5. How can a Kalman filter estimate a dynamic hedge ratio?

A. Regress y_t on x_t with state $[\alpha_t, \beta_t]$, $F = I$ and the observation matrix $H_t = [1, x_t]$. The state follows a random walk with covariance Q (how fast the hedge ratio moves), and R is the spread noise. Each day the filter predicts y, compares with

the actual price, and moves $\hat{\beta}$ in proportion to the innovation and the gain. It yields a hedge ratio, its uncertainty, and the standardised innovation as a trading signal.

Q6. Why can a state space model perform poorly after a market regime change?
A. Its parameters (F, H, Q, R) and noise assumptions were set or estimated on the old regime. After a change the innovations are larger and persistent; a filter with small Q reacts too slowly, and one with large Q chases noise. It may also absorb the divergence into the state (as in Chapter 8). A linear Gaussian filter has no mechanism to recognise a discrete regime, so switching models, monitoring of innovations and re-estimation rules are needed.

Q7. Why does multiplying both Q and R by a constant not change the filtered states?
A. The gain depends on P/(P + R) and P scales with Q, so the ratio q = Q/R alone fixes K. P and S scale with the constant and the log-likelihood changes, but the state estimates do not (verified in Section 4.5). This also shows that Q and R are only separately identifiable through the likelihood.

Q8. Which parameters can be estimated and how?
A. Variances in Q and R (and elements of F) are estimated by maximum likelihood using the prediction-error decomposition, or by EM. Identification can be weak: the likelihood for Q was very flat in our example, and a persistent spread with white observation noise pushes ML toward an over-flexible state. Practitioners often fix some parameters from priors and validate out of sample.

Q9. When do you need an EKF, UKF or particle filter, and is stochastic volatility linear Gaussian?
A. When the dynamics or observation are nonlinear, or the noise is non-Gaussian. EKF linearises; UKF uses sigma points; particle filters use samples and handle general models. Stochastic volatility is nonlinear in the original observation, and the log-squared-return transformation produces non-Gaussian errors, so a standard Kalman filter gives only a quasi-likelihood approximation.

Q10. Why use the standardised innovation as a pairs-trading signal, and what can go wrong?
A. $e_t/\sqrt{S_t}$ is the forecast surprise in units of its predicted standard deviation, so its scale does not depend on the price level or volatility regime. Problems: if the filter adapts too quickly it absorbs divergences and the signal vanishes; the innovations can be autocorrelated and non-Gaussian, so thresholds are not calibrated probabilities; and after a break the signal is persistently large.

10.2 Five numerical, mathematical and coding questions

N1. Scalar update. F = H = 1, Q = 1, R = 2, $\hat{x}_{0|0} = 10$, $P_{0|0} = 4$, $y_1 = 12$. Compute $\hat{x}_{1|1}$, $P_{1|1}$.
A. $P_{1|0} = 5$, S = 7, K = 5/7 = 0.7143, e = 2, so $\hat{x}_{1|1} = 10 + 0.7143 \times 2 = 11.4286$ and $P_{1|1} = (1 - 0.7143) \times 5 = 1.4286$ (Table 5).

N2. Steady-state gain. For the local-level model with Q = 0.25 and R = 4, find the steady-state gain and the equivalent EMA span.
A. Solve $P^2 - QP - QR = 0$ for the prediction variance: $P = (0.25 + \sqrt{(0.0625 + 4)})/2 = 1.1328$. K = P/(P + R) = 0.2207. An EMA with smoothing constant α has span $2/\alpha - 1$, so the span is 8.1 days; the weight on observations older than k steps is $(1 - K)^k$, which falls below 5% after 12.0 observations. The simulated filter's final gain was 0.2207, matching.

N3. Forecast interval. At the end of the sample the filtered level is 100.54 with $P_{T|T} = 0.883$. Give the 95% interval for the observation 5 days ahead (Q = 0.25, R = 4).
A. Variance = P + 5Q + R = 0.883 + 1.25 + 4 = 6.133; standard deviation 2.476 (Table 7). Interval: 100.54 ± 1.96 × 2.476 = [95.69, 105.40].

N4. Scale invariance. Show that replacing (Q, R) by (10Q, 10R) leaves the filtered state unchanged, and verify numerically.
A. With $(Q, R, P_0) \to (10Q, 10R, 10P_0)$, every P and S scales by 10 and $K = PH^T/S$ is unchanged, so $\hat{x}$ is unchanged while the log-likelihood shifts. Running the filter of Section 3.3 both ways gave a maximum absolute difference of 0: `np.abs(kf10["x_f"] - kf["x_f"]).max()`.

N5. Coding. Given the filter output kf and log prices y, x, write the lines that create a position series with no look-ahead, and name two places where look-ahead could creep in.
A. Signal and timing: `z = kf['e'] / np.sqrt(kf['S']); pos = positions(z); pnl[1:] = pos[:-1] * (np.diff(y) - beta_f[:-1] * np.diff(x))`, i.e. the position set at t earns the $t \to t + 1$ return with $\hat{\beta}_{t|t}$. Look-ahead creeps in (i) by using the smoothed state or hedge ratio instead of x_f, and (ii) by multiplying the position at t with the return from t − 1 to t, or fitting Q and R on the full sample including the test period.

10.3 Formula sheet

Table 17. Key formulas

Quantity	Formula
State equation	$x_t = F_t x_{t-1} + B_t u_t + w_t,\ \ w_t \sim N(0, Q_t)$
Observation equation	$y_t = H_t x_t + D_t u_t + v_t,\ \ v_t \sim N(0, R_t)$
State prediction	$\hat{x}_{t\|t-1} = F_t \hat{x}_{t-1\|t-1}$
Covariance prediction	$P_{t\|t-1} = F_t P_{t-1\|t-1} F_t^T + Q_t$
Innovation and its variance	$e_t = y_t - H_t \hat{x}_{t\|t-1},\ \ S_t = H_t P_{t\|t-1} H_t^T + R_t$
Kalman gain	$K_t = P_{t\|t-1} H_t^T S_t^{-1}$
Update	$\hat{x}_{t\|t} = \hat{x}_{t\|t-1} + K_t e_t;\ \ P_{t\|t} = (I - K_t H_t) P_{t\|t-1}$
Scalar gain (H = 1)	$K = P_{t\|t-1}/(P_{t\|t-1} + R)$
Local-level steady state	$P^2 - QP - QR = 0,\ \ P = (Q + \sqrt{(Q^2 + 4QR)})/2,\ \ K = P/(P + R)$
RTS smoother	$\hat{x}_{t\|T} = \hat{x}_{t\|t} + J_t(\hat{x}_{t+1\|T} - \hat{x}_{t+1\|t}),\ \ J_t = P_{t\|t} F^T P_{t+1\|t}^{-1}$
h-step forecast	$\hat{x}_{t+h\|t} = F^h \hat{x}_{t\|t}$; local level: $\mathrm{Var} = P_{t\|t} + hQ$ (+ R for y)
Log-likelihood	$\log L = -\tfrac{1}{2} \Sigma\, [\log(2\pi S_t) + e_t^2/S_t]$
Standardised innovation	$z_t = e_t/\sqrt{S_t} \sim N(0, 1)$ if the model is correct
Dynamic regression	$y_t = H_t x_t + v_t,\ \ H_t = [1, x_t],\ \ \theta_t = [\alpha_t, \beta_t]^T,\ \ F = I$
Local linear trend	$\mu_t = \mu_{t-1} + \beta_{t-1} + \xi_t;\ \ \beta_t = \beta_{t-1} + \zeta_t;\ \ y_t = \mu_t + \varepsilon_t$

10.4 Glossary

Table 18. Glossary

Term	Meaning
Control input	Known external variable entering the state or observation equation.
Diffuse prior	A prior with very large variance so the first data dominate.
Expectation–maximisation (EM)	Iterative estimation that alternates smoothing and parameter updates.
Filtering	Estimating the state at t using data up to t.
Forecast uncertainty	Variance of a future state or observation given current information.
Hidden Markov model	Model with a discrete hidden state following a Markov chain.
Innovation	One-step-ahead forecast error $e_t = y_t - \hat{y}_{t\|t-1}$.
Kalman gain	Weight placed on the innovation when updating the state.
Latent state	Unobserved variable whose dynamics we model.
Local-level model	Random-walk state observed with noise.
Observation noise (R)	Measurement error between state and data.
Process noise (Q)	Random shocks driving state changes.
Look-ahead bias	Using information not available at decision time.
Particle filter	Sequential Monte Carlo filter for nonlinear / non-Gaussian models.
Smoothing	Estimating the state at t using data beyond t.
Standardised innovation	$e_t/\sqrt{S_t}$; unit variance if the model is right.
Structural break	Permanent change in a model's parameters or relationships.
Time-varying parameter model	Regression whose coefficients evolve over time.

10.5 Further reading and references

[1] Durbin, J. and Koopman, S. J. (2012). *Time Series Analysis by State Space Methods*, 2nd ed. Oxford University Press. The definitive treatment of linear Gaussian SSMs.

[2] Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.

[3] Shumway, R. H. and Stoffer, D. S. *Time Series Analysis and Its Applications: With R Examples*. Springer (Chapter on state space models; EM).

[4] Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press (the Kalman filter and regime-switching models).

[5] Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.

[6] Rauch, H. E., Tung, F. and Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8), 1445–1450.

[7] Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2), 357–384.

[8] Harvey, A., Ruiz, E. and Shephard, N. (1994). Multivariate stochastic variance models. *Review of Economic Studies*, 61(2), 247–264.

[9] Gordon, N. J., Salmond, D. J. and Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F*, 140(2), 107–113.

[10] Julier, S. J. and Uhlmann, J. K. (2004). Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, 92(3), 401–422.

[11] Engle, R. F. and Granger, C. W. J. (1987). Co-integration and error correction. *Econometrica*, 55(2), 251–276.

[12] Chan, E. P. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley (Kalman filter pairs trading).

[13] Bailey, D. H., Borwein, J. M., López de Prado, M. and Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism. *Notices of the AMS*, 61(5), 458–471.

[14] Seabold, S. and Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*. Documentation of the state space module: statsmodels.org/stable/statespace.html

[15] FilterPy documentation (R. Labbe): filterpy.readthedocs.io. NumPy, SciPy and Matplotlib documentation: numpy.org/doc, docs.scipy.org/doc, matplotlib.org/stable.

Edition years and chapter references were not checked beyond the details above; consult current editions before citing.